

Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity

William Fedus*

LIAMFEDUS@GOOGLE.COM

Barret Zoph*

BARRETZOPH@GOOGLE.COM

Noam Shazeer

NOAM@GOOGLE.COM

Google, Mountain View, CA 94043, USA

Editor: Alexander Clark

Abstract

In deep learning, models typically reuse the same parameters for all inputs. Mixture of Experts (MoE) models defy this and instead select *different* parameters for each incoming example. The result is a sparsely-activated model—with an outrageous number of parameters—but a constant computational cost. However, despite several notable successes of MoE, widespread adoption has been hindered by complexity, communication costs, and training instability. We address these with the introduction of the Switch Transformer. We simplify the MoE routing algorithm and design intuitive improved models with reduced communication and computational costs. Our proposed training techniques mitigate the instabilities, and we show large sparse models may be trained, for the first time, with lower precision (bfloat16) formats. We design models based off T5-Base and T5-Large (Raffel et al., 2019) to obtain up to 7x increases in pre-training speed with the same computational resources. These improvements extend into multilingual settings where we measure gains over the mT5-Base version across all 101 languages. Finally, we advance the current scale of language models by pre-training up to trillion parameter models on the “Colossal Clean Crawled Corpus”, and achieve a 4x speedup over the T5-XXL model.¹²

Keywords: mixture-of-experts, natural language processing, sparsity, large-scale machine learning, distributed computing

*. Equal contribution.

1. JAX code for Switch Transformer and all model checkpoints are available at <https://github.com/google-research/t5x>
2. Tensorflow code for Switch Transformer is available at https://github.com/tensorflow/mesh/blob/master/mesh_tensorflow/transformer/moe.py

Contents

1	Introduction	3
2	Switch Transformer	4
2.1	Simplifying Sparse Routing	5
2.2	Efficient Sparse Routing	6
2.3	Putting It All Together: The Switch Transformer	8
2.4	Improved Training and Fine-Tuning Techniques	8
3	Scaling Properties	11
3.1	Scaling Results on a Step-Basis	12
3.2	Scaling Results on a Time-Basis	13
3.3	Scaling Versus a Larger Dense Model	13
4	Downstream Results	14
4.1	Fine-Tuning	14
4.2	Distillation	16
4.3	Multilingual Learning	17
5	Designing Models with Data, Model, and Expert-Parallelism	18
5.1	Data Parallelism	20
5.2	Model Parallelism	20
5.3	Model and Data Parallelism	21
5.4	Expert and Data Parallelism	22
5.5	Expert, Model and Data Parallelism	22
5.6	Towards Trillion Parameter Models	22
6	Related Work	24
7	Discussion	25
8	Future Work	26
9	Conclusion	27
A	Switch for Attention	27
B	Preventing Token Dropping with <i>No-Token-Left-Behind</i>	29
C	Encouraging Exploration Across Experts	29
D	Switch Transformers in Lower Compute Regimes	29
E	Relation of Upstream to Downstream Model Performance	32
F	Pseudo Code for Switch Transformers	33

1. Introduction

Large scale training has been an effective path towards flexible and powerful neural language models (Radford et al., 2018; Kaplan et al., 2020; Brown et al., 2020). Simple architectures—backed by a generous computational budget, data set size and parameter count—surpass more complicated algorithms (Sutton, 2019). An approach followed in Radford et al. (2018); Raffel et al. (2019); Brown et al. (2020) expands the model size of a densely-activated Transformer (Vaswani et al., 2017). While effective, it is also extremely computationally intensive (Strubell et al., 2019). Inspired by the success of model scale, but seeking greater computational efficiency, we instead propose a *sparsely-activated* expert model: the Switch Transformer. In our case the sparsity comes from activating a *subset* of the neural network weights for each incoming example.

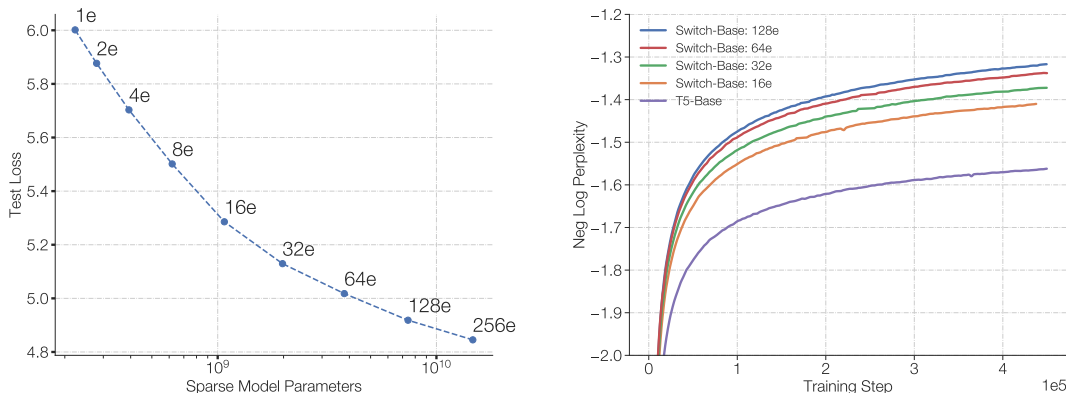


Figure 1: Scaling and sample efficiency of Switch Transformers. Left Plot: Scaling properties for increasingly sparse (more experts) Switch Transformers. Right Plot: Negative log perplexity comparing Switch Transformers to T5 (Raffel et al., 2019) models using the same compute budget.

Sparse training is an active area of research and engineering (Gray et al., 2017; Gale et al., 2020), but as of today, machine learning libraries and hardware accelerators still cater to dense matrix multiplications. To have an efficient sparse algorithm, we start with the Mixture-of-Expert (MoE) paradigm (Jacobs et al., 1991; Jordan and Jacobs, 1994; Shazeer et al., 2017), and simplify it to yield training stability and computational benefits. MoE models have had notable successes in machine translation (Shazeer et al., 2017, 2018; Lepikhin et al., 2020), however, widespread adoption is hindered by complexity, communication costs, and training instabilities.

We address these issues, and then go beyond translation, to find that these class of algorithms are broadly valuable in natural language. We measure superior scaling on a diverse set of natural language tasks and across three regimes in NLP: pre-training, fine-tuning and multi-task training. While this work focuses on scale, we also show that the Switch Transformer architecture not only excels in the domain of supercomputers, but is

beneficial even with only a few computational cores. Further, our large sparse models can be distilled (Hinton et al., 2015) into small dense versions while preserving 30% of the sparse model quality gain. Our contributions are the following:

- The Switch Transformer architecture, which simplifies and improves over Mixture of Experts.
- Scaling properties and a benchmark against the strongly tuned T5 model (Raffel et al., 2019) where we measure 7x+ pre-training speedups while still using the same FLOPS per token. We further show the improvements hold even with limited computational resources, using as few as two experts.
- Successful distillation of sparse pre-trained and specialized fine-tuned models into small dense models. We reduce the model size by up to 99% while preserving 30% of the quality gains of the large sparse teacher.
- Improved pre-training and fine-tuning techniques: **(1)** selective precision training that enables training with lower bfloat16 precision **(2)** an initialization scheme that allows for scaling to a larger number of experts and **(3)** increased expert regularization that improves sparse model fine-tuning and multi-task training.
- A measurement of the pre-training benefits on multilingual data where we find a universal improvement across all 101 languages and with 91% of languages benefiting from 4x+ speedups over the mT5 baseline (Xue et al., 2020).
- An increase in the scale of neural language models achieved by efficiently combining data, model, and expert-parallelism to create models with up to a trillion parameters. These models improve the pre-training speed of a strongly tuned T5-XXL baseline by 4x.

2. Switch Transformer

The guiding design principle for Switch Transformers is to maximize the parameter count of a Transformer model (Vaswani et al., 2017) in a simple and computationally efficient way. The benefit of scale was exhaustively studied in Kaplan et al. (2020) which uncovered power-law scaling with model size, data set size and computational budget. Importantly, this work advocates training large models on relatively small amounts of data as the computationally optimal approach.

Heeding these results, we investigate a fourth axis: increase the *parameter count* while keeping the floating point operations (FLOPs) per example constant. Our hypothesis is that the parameter count, independent of total computation performed, is a separately important axis on which to scale. We achieve this by designing a sparsely activated model that efficiently uses hardware designed for dense matrix multiplications such as GPUs and TPUs. Our work here focuses on TPU architectures, but these class of models may be similarly trained on GPU clusters. In our distributed training setup, our sparsely activated layers split *unique* weights on different devices. Therefore, the weights of the model increase with the number of devices, all while maintaining a manageable memory and computational footprint on each device.

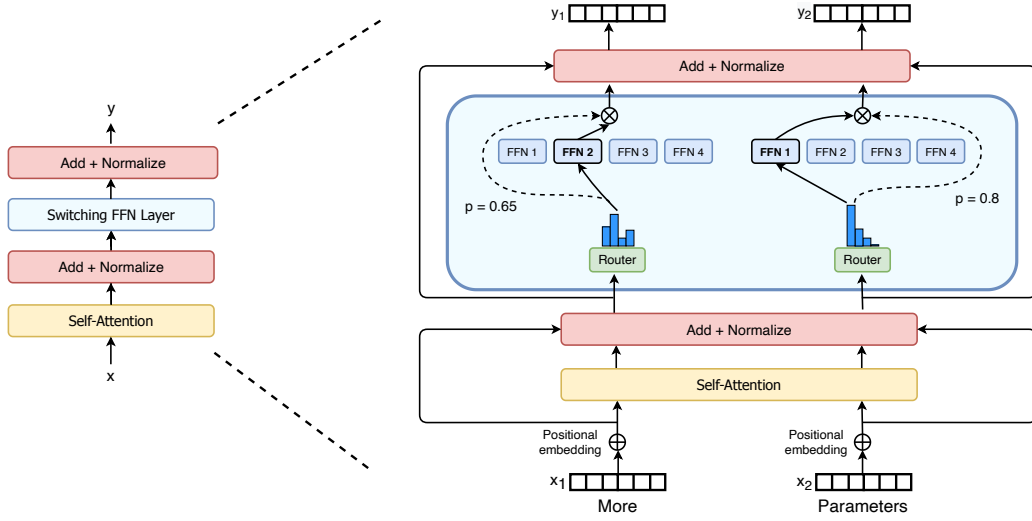


Figure 2: Illustration of a Switch Transformer encoder block. We replace the dense feed forward network (FFN) layer present in the Transformer with a sparse Switch FFN layer (light blue). The layer operates independently on the tokens in the sequence. We diagram two tokens (x_1 = “More” and x_2 = “Parameters” below) being routed (solid lines) across four FFN experts, where the router independently routes each token. The switch FFN layer returns the output of the selected FFN multiplied by the router gate value (dotted-line).

2.1 Simplifying Sparse Routing

Mixture of Expert Routing. Shazeer et al. (2017) proposed a natural language Mixture-of-Experts (MoE) layer which takes as an input a token representation x and then routes this to the best determined top- k experts, selected from a set $\{E_i(x)\}_{i=1}^N$ of N experts. The router variable W_r produces logits $h(x) = W_r \cdot x$ which are normalized via a softmax distribution over the available N experts at that layer. The gate-value for expert i is given by,

$$p_i(x) = \frac{e^{h(x)_i}}{\sum_j^N e^{h(x)_j}}. \quad (1)$$

The top- k gate values are selected for routing the token x . If $\mathcal{T}$ is the set of selected top- k indices then the output computation of the layer is the linearly weighted combination of each expert’s computation on the token by the gate value,

$$y = \sum_{i \in \mathcal{T}} p_i(x) E_i(x). \quad (2)$$

Switch Routing: Rethinking Mixture-of-Experts. Shazeer et al. (2017) conjectured that routing to $k > 1$ experts was necessary in order to have non-trivial gradients to the routing functions. The authors intuited that learning to route would not work without the ability to compare at least two experts. Ramachandran and Le (2018) went further to

study the top- k decision and found that higher k -values in lower layers in the model were important for models with many routing layers. Contrary to these ideas, we instead use a simplified strategy where we route to only a *single* expert. We show this simplification preserves model quality, reduces routing computation and performs better. This $k = 1$ routing strategy is later referred to as a Switch layer. Note that for both MoE and Switch Routing, the gate value $p_i(x)$ in Equation 2 permits differentiability of the router.

The benefits for the Switch layer are three-fold: **(1)** The router computation is reduced as we are only routing a token to a single expert. **(2)** The batch size (expert capacity) of each expert can be at least halved since each token is only being routed to a single expert.³ **(3)** The routing implementation is simplified and communication costs are reduced. Figure 3 shows an example of routing with different expert capacity factors.

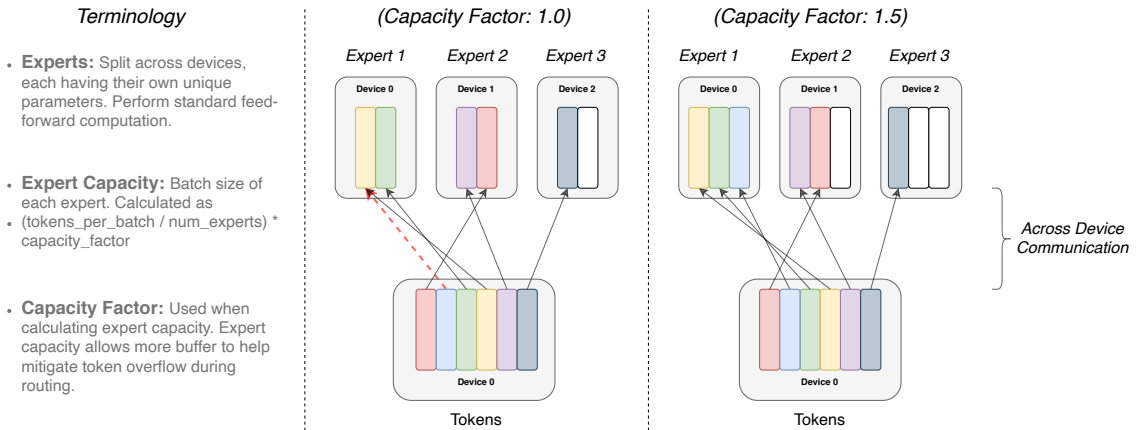


Figure 3: Illustration of token routing dynamics. Each expert processes a fixed batch-size of tokens modulated by the *capacity factor*. Each token is routed to the expert with the highest router probability, but each expert has a fixed batch size of $(\text{total_tokens} / \text{num_experts}) \times \text{capacity_factor}$. If the tokens are unevenly dispatched then certain experts will overflow (denoted by dotted red lines), resulting in these tokens not being processed by this layer. A larger capacity factor alleviates this overflow issue, but also increases computation and communication costs (depicted by padded white/empty slots).

2.2 Efficient Sparse Routing

We use Mesh-Tensorflow (MTF) (Shazeer et al., 2018) which is a library, with similar semantics and API to Tensorflow (Abadi et al., 2016) that facilitates efficient distributed data and model parallel architectures. It does so by abstracting the physical set of cores to a logical mesh of processors. Tensors and computations may then be sharded per named dimensions, facilitating easy partitioning of models across dimensions. We design our model with TPUs in mind, which require statically declared sizes. Below we describe our distributed Switch Transformer implementation.

3. See Section 2.2 for a technical description.

Distributed Switch Implementation. All of our tensor shapes are statically determined at compilation time, but our computation is *dynamic* due to the routing decisions at training and inference. Because of this, one important technical consideration is how to set the *expert capacity*. The expert capacity—the number of tokens each expert computes—is set by evenly dividing the number of tokens in the batch across the number of experts, and then further expanding by a *capacity factor*,

$$\text{expert capacity} = \left(\frac{\text{tokens per batch}}{\text{number of experts}} \right) \times \text{capacity factor}. \quad (3)$$

A capacity factor greater than 1.0 creates additional buffer to accommodate for when tokens are not perfectly balanced across experts. If too many tokens are routed to an expert (referred to later as dropped tokens), computation is skipped and the token representation is passed directly to the next layer through the residual connection. Increasing the expert capacity is not without drawbacks, however, since high values will result in wasted computation and memory. This trade-off is explained in Figure 3. Empirically we find ensuring lower rates of dropped tokens are important for the scaling of sparse expert-models. Throughout our experiments we didn’t notice any dependency on the number of experts for the number of tokens dropped (typically $< 1\%$). Using the auxiliary load balancing loss (next section) with a high enough coefficient ensured good load balancing. We study the impact that these design decisions have on model quality and speed in Table 1.

A Differentiable Load Balancing Loss. To encourage a balanced load across experts we add an auxiliary loss (Shazeer et al., 2017, 2018; Lepikhin et al., 2020). As in Shazeer et al. (2018); Lepikhin et al. (2020), Switch Transformers simplifies the original design in Shazeer et al. (2017) which had separate load-balancing and importance-weighting losses. For each Switch layer, this auxiliary loss is added to the total model loss during training. Given N experts indexed by $i = 1$ to N and a batch $\mathcal{B}$ with T tokens, the auxiliary loss is computed as the scaled dot-product between vectors f and P ,

$$\text{loss} = \alpha \cdot N \cdot \sum_{i=1}^N f_i \cdot P_i \quad (4)$$

where f_i is the fraction of tokens dispatched to expert i ,

$$f_i = \frac{1}{T} \sum_{x \in \mathcal{B}} \mathbb{1}\{\text{argmax } p(x) = i\} \quad (5)$$

and P_i is the fraction of the router probability allocated for expert i ,²

$$P_i = \frac{1}{T} \sum_{x \in \mathcal{B}} p_i(x). \quad (6)$$

Since we seek uniform routing of the batch of tokens across the N experts, we desire both vectors to have values of $1/N$. The auxiliary loss of Equation 4 encourages uniform routing since it is minimized under a uniform distribution. The objective can also be differentiated as

2. A potential source of confusion: $p_i(x)$ is the probability of routing token x to expert i . P_i is the probability fraction to expert i across *all tokens* in the batch $\mathcal{B}$.

the P -vector is differentiable, but the f -vector is not. The final loss is multiplied by expert count N to keep the loss constant as the number of experts varies since under uniform routing $\sum_{i=1}^N (f_i \cdot P_i) = \sum_{i=1}^N (\frac{1}{N} \cdot \frac{1}{N}) = \frac{1}{N}$. Finally, a hyper-parameter α is a multiplicative coefficient for these auxiliary losses; throughout this work we use an $\alpha = 10^{-2}$ which was sufficiently large to ensure load balancing while small enough to not overwhelm the primary cross-entropy objective. We swept hyper-parameter ranges of α from 10^{-1} to 10^{-5} in powers of 10 and found 10^{-2} balanced load quickly without interfering with training loss.

2.3 Putting It All Together: The Switch Transformer

Our first test of the Switch Transformer starts with pre-training on the ‘‘Colossal Clean Crawled Corpus’’ (C4), introduced in (Raffel et al., 2019). For our pre-training objective, we use a masked language modeling task (Taylor, 1953; Fedus et al., 2018; Devlin et al., 2018) where the model is trained to predict missing tokens. In our pre-training setting, as determined in Raffel et al. (2019) to be optimal, we drop out 15% of tokens and then replace the masked sequence with a single sentinel token. To compare our models, we record the negative log perplexity.⁴ Throughout all tables in the paper, $\uparrow$ indicates that a higher value for that metric is better and vice-versa for $\downarrow$. A comparison of all the models studied in this work are in Table 9.

A head-to-head comparison of the Switch Transformer and the MoE Transformer is presented in Table 1. Our Switch Transformer model is FLOP-matched to ‘T5-Base’ (Raffel et al., 2019) (same amount of computation per token is applied). The MoE Transformer, using top-2 routing, has two experts which each apply a separate FFN to each token and thus its FLOPS are larger. All models were trained for the same number of steps on identical hardware. Note that the MoE model going from capacity factor 2.0 to 1.25 actually slows down (840 to 790) in the above experiment setup, which is unexpected.⁵

We highlight three key findings from Table 1: **(1)** Switch Transformers outperform both carefully tuned dense models and MoE Transformers on a speed-quality basis. For a fixed amount of computation and wall-clock time, Switch Transformers achieve the best result. **(2)** The Switch Transformer has a smaller computational footprint than the MoE counterpart. If we increase its size to match the training speed of the MoE Transformer, we find this outperforms all MoE and Dense models on a per step basis as well. **(3)** Switch Transformers perform better at lower capacity factors (1.0, 1.25). Smaller expert capacities are indicative of the scenario in the large model regime where model memory is very scarce and the capacity factor will want to be made as small as possible.

2.4 Improved Training and Fine-Tuning Techniques

Sparse expert models may introduce training difficulties over a vanilla Transformer. Instability can result because of the hard-switching (routing) decisions at each of these layers. Further, low precision formats like bfloat16 (Wang and Kanwar, 2019) can exacerbate issues

4. We use log base-e for this metric so the units are nats.

5. Note that speed measurements are both a function of the algorithm and the implementation details. Switch Transformer reduces the necessary computation relative to MoE (algorithm), but the final speed differences are impacted by low-level optimizations (implementation).

Model	Capacity Factor	Quality after 100k steps (↑) (Neg. Log Perp.)	Time to Quality Threshold (↓) (hours)	Speed (↑) (examples/sec)
T5-Base	—	-1.731	Not achieved [†]	1600
T5-Large	—	-1.550	131.1	470
MoE-Base	2.0	-1.547	68.7	840
Switch-Base	2.0	-1.554	72.8	860
MoE-Base	1.25	-1.559	80.7	790
Switch-Base	1.25	-1.553	65.0	910
MoE-Base	1.0	-1.572	80.1	860
Switch-Base	1.0	-1.561	62.8	1000
Switch-Base+	1.0	-1.534	67.6	780

Table 1: Benchmarking Switch versus MoE. Head-to-head comparison measuring per step and per time benefits of the Switch Transformer over the MoE Transformer and T5 dense baselines. We measure quality by the negative log perplexity and the time to reach an arbitrary chosen quality threshold of Neg. Log Perp.=-1.50. All MoE and Switch Transformer models use 128 experts, with experts at every other feed-forward layer. For Switch-Base+, we increase the model size until it matches the speed of the MoE model by increasing the model hidden-size from 768 to 896 and the number of heads from 14 to 16. All models are trained with the same amount of computation (32 cores) and on the same hardware (TPUv3). Further note that all our models required pre-training beyond 100k steps to achieve our level threshold of -1.50. [†] T5-Base did not achieve this negative log perplexity in the 100k steps the models were trained.

in the softmax computation for our router. We describe training difficulties here and the methods we use to overcome them to achieve stable and scalable training.

Selective precision with large sparse models. Model instability hinders the ability to train using efficient bfloat16 precision, and as a result, Lepikhin et al. (2020) trains with float32 precision throughout their MoE Transformer. However, we show that by instead *selectively casting* to float32 precision within a localized part of the model, stability may be achieved, without incurring expensive communication cost of float32 tensors. This technique is inline with modern mixed precision training strategies where certain parts of the model and gradient updates are done in higher precision Micikevicius et al. (2017). Table 2 shows that our approach permits nearly equal speed to bfloat16 training while conferring the training stability of float32.

To achieve this, we cast the router input to float32 precision. The router function takes the tokens as input and produces the dispatch and combine tensors used for the selection and recombination of expert computation (refer to Code Block 15 in the Appendix for details). Importantly, the float32 precision is only used *within* the body of the router function—on computations local to that device. Because the resulting dispatch and combine tensors are recast to bfloat16 precision at the end of the function, no expensive float32 tensors